means you don't have to search for log messages and can get away with less indexing overhead.

Now, let's see how we can put that knowledge into practice, starting with emitting spans in our example app.

2.5.1 Instrumentation

Listing 2.10 shows the relevant lines of our `echo` service in the context of instrumentation with traces. Note that we are using OpenTelemetry here to emit the spans, which we will discuss in greater detail in chapter 4.

Listing 2.10 Example Go program, instrumented to emit traces

```
func handleEcho(w http.ResponseWriter, r *http.Request) {
    message := r.URL.Query().Get("message")
    log.WithFields(log.Fields{
        "service": "echo",
    }).Info("Got input ", message)
    ctx, cancel := context.WithCancel(context.Background())
    defer cancel()
    tr := otel.Tracer("http-api")
    _, span := tr.Start(ctx, "echo")
    defer span.End()
    if rand.Intn(100) > 90 {
        log.WithFields(log.Fields{
            "service": "echo",
        }).Error("Something really bad happened :(")
        invokes.WithLabelValues(
            strconv.Itoa(http.StatusInternalServerError)).Inc()
        span.SetAttributes(
            attribute.Key("http-status-code").String(
                strconv.Itoa(http.StatusInternalServerError)))
        w.WriteHeader(500)
        return
    }
    invokes.WithLabelValues(
        strconv.Itoa(http.StatusOK)).Inc()
    span.SetAttributes(
        attribute.Key("http-status-code").String(
            strconv.Itoa(http.StatusOK)))
    fmt.Fprintf(w, message)
}

func main() {
    rand.Seed(time.Now().UnixNano())
    registry.MustRegister(invokes)
    log.SetFormatter(&log.JSONFormatter{})
    http.HandleFunc("/echo", handleEcho)
```

Starts a new span when invoked

Sets the error code as an attribute (signal metadata) of the span

Sets success as an attribute (signal metadata) of the span